“PF PROJECT PROPOSAL”

FINAL PROJECT REPORT:(PROGRAMMING FUNDAMENTALS):

University Name: Fast NUCES.

Department: Department of Computer Science.

Course: Programming Fundamentals.

Project Title: Tetris.

Submitted By : Muhammad Umair (25K-0510) And Muhammad Abdul Moeed (25K-0659).

Submitted To: Miss Hafiza Bushra.

Semester: Fall 2025.

Date of Submission: 23/Nov/2025.

Abstract:

This project is about making a complete, functioning version of the game Tetris using the C Programming Language. Running in the console window, it implements the basic concepts of programming: arrays, structures, loops, conditional statements, and simple game logic. The game would run completely in the terminal, making use of arrays, loops, structures, and real-time keyboard input to simulate game-playing. While the blocks fall, the player has to position them such that they complete the horizontal lines; once a line is complete, it disappears and earns points for the player. This project will be a good introduction to game development, logic, state management, and event handling within the scope of a somewhat simple text-based interface.

INTRODUCTION:

One of the famous puzzle games is Tetris, and its implementation in C gives a chance to understand how real-time programs work. The goal of this project was to create a fully playable version of Tetris running on the console, using only basic programming tools. This project makes use of a 20×10 grid display, seven standard shapes, and real-time user controls to recreate classic gameplay. It puts into practice core concepts such as arrays, loops, conditional statements, and structures in handling falling shapes, player movement, rotation, updating the grid, and keeping the score. This project provides not only the recreation of classic gameplay but also hands-on experience with how interactive programs are designed and how different parts of a program interact with each other to create a complete system.

2.OBJECTIVES:

1. To design and implement a classic Tetris game using C language.

2. To generate and control falling tetrimino shapes.

3. To apply real-time keyboard input handling and collision detection.

4. To clear lines and update scores dynamically.

5. To reinforce understanding of arrays, loops, functions, structures, and game logic.

3.SYSTEM DESIGN:

3.1 Program Overview

The game continuously updates the grid as pieces fall. Players can control the pieces using actions like moving left, right, down, or rotating. Once a piece lands, it becomes fixed in place, and a new piece appears. This process continues until the grid is completely filled, triggering a game-over state.

3.2 Flowchart

Start → Initialize Grid → Generate New Piece → Display Grid → Handle User Input → Move/Rotate Piece → Check for Collision → Fix Piece → Clear Completed Lines → Update Score → Spawn New Piece → Game Over → End

3.3 Input & Output

Input:

Keyboard Controls:

- **A** (Move Left)
- **D** (Move Right)
- **S** (Drop Faster)
- **W** (Rotate Piece)
-

Output:

- Real-time game grid display
- Falling piece visualization
- Updated score after line clears
- “Game Over” message when the grid is full

4.IMPLEMENTATION:

Language: C

Platform: Windows (using conio.h and windows.h).

KEY FEATURES:

- Real-time tetrimino movement
- Block rotation
- Collision detection
- Line clearing and scoring
- Game Over detection
- Random shape generation

Important Code Functions

- newpiece().
- sector() (game display)
- checkbump() (collision detection)
- PieceFixed()
- VanishLines()
- SpinPiece() (rotation)

5. TESTING AND RESULTS:

TEST NO	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
1	Tetrimino reaches bottom	Shape becomes fixed	Work correctly	✓
2	Line completely filled	Line disappear and score increases	Works correctly	✓
3	Rotate Shape near wall	Rotation allowed only if no collision	Works correctly	✓
4	Grid fills completely	Game Over displayed	Works correctly	✓

6.CONCLUSION, LIMITATION AND REFERENCES:

Conclusion:

The Console-Based Tetris Game successfully demonstrates real-time interactive programming using C. The project strengthened understanding of arrays, structures, loops, collision handling, and basic game physics. This project shows how classical games can be built using fundamental concepts without advanced libraries.

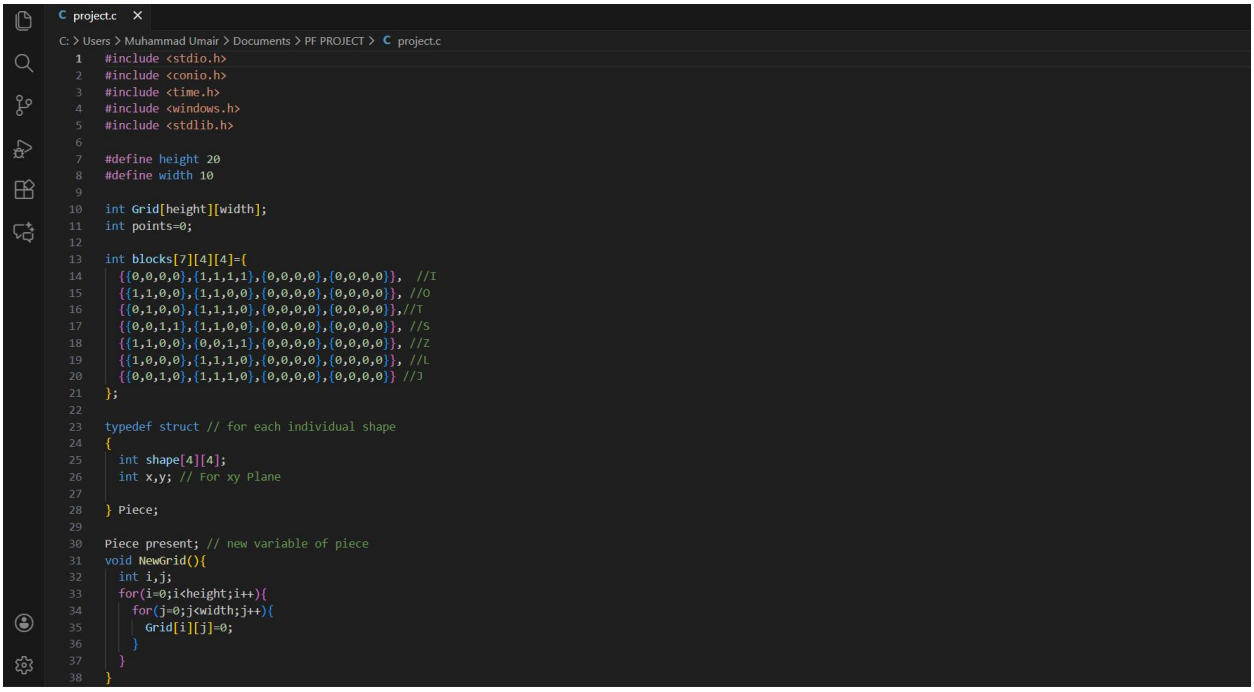
Limitations:

1. Only works on Windows due to conio.h and windows.h.
2. No sound or color graphics.
3. No saving of high scores.

References:

1. https://tetris.com
2. Saltsman, A. (2010). *Game Mechanics: Advanced Game Design*
3. Game Developers Conference (GDC) Talks – "Tetris & Puzzle Game Design.".

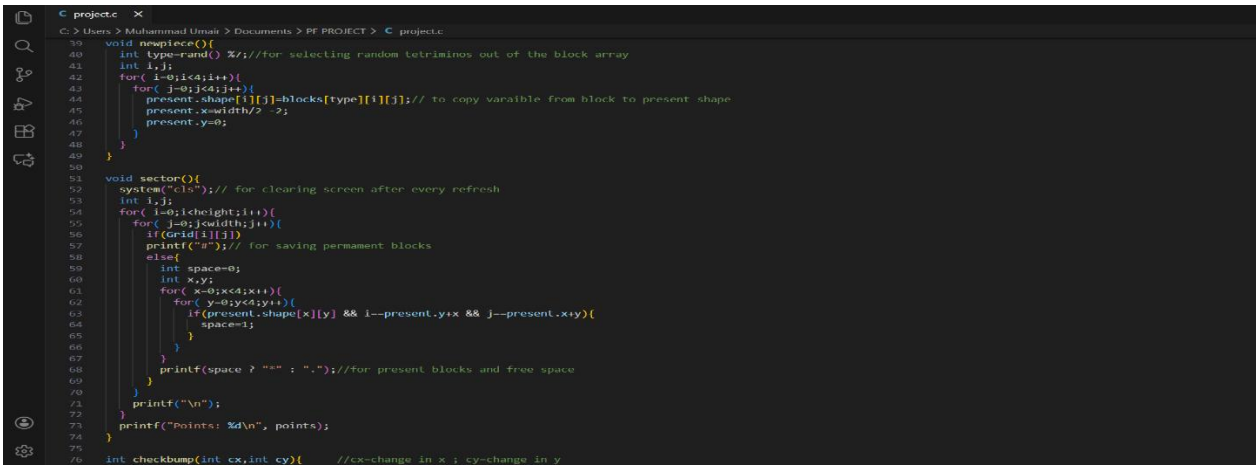
CODE SNIPPETS:



project.c

C: > Users > Muhammad Umair > Documents > PF PROJECT > project.c

```c
#include <stdio.h>
#include <conio.h>
#include <time.h>
#include <windows.h>
#include <stdlib.h>

#define height 20
#define width 10

int Grid[height][width];
int points=0;

int blocks[7][4][4]={
  {{0,0,0,0},{1,1,1,1},{0,0,0,0},{0,0,0,0}},  //I
  {{1,1,0,0},{1,1,0,0},{0,0,0,0},{0,0,0,0}}, //O
  {{0,1,0,0},{1,1,1,0},{0,0,0,0},{0,0,0,0}},//T
  {{0,0,1,1},{1,1,0,0},{0,0,0,0},{0,0,0,0}}, //S
  {{1,1,0,0},{0,0,1,1},{0,0,0,0},{0,0,0,0}}, //Z
  {{1,0,0,0},{1,1,1,0},{0,0,0,0},{0,0,0,0}}, //L
  {{0,0,1,0},{1,1,1,0},{0,0,0,0},{0,0,0,0}} //J
};

typedef struct // for each individual shape
{
  int shape[4][4];
  int x,y; // For xy Plane

} Piece;

Piece present; // new variable of piece
void NewGrid(){
  int i,j;
  for(i=0;i<height;i++){
    for(j=0;j<width;j++){
      Grid[i][j]=0;
    }
  }
}
```



project.c

C: > Users > Muhammad Umair > Documents > PF PROJECT > project.c

```c
void newpiece(){
  int type=rand() %7;//for selecting random tetriminos out of the block array
  int i,j;
  for( i=0;i<4;i++){
    for( j=0;j<4;j++){
      present.shape[i][j]=blocks[type][i][j];// to copy varaible from block to present shape
      present.x=width/2 -2;
      present.y=0;
    }
  }
}

void sector(){
  system("cls");// for clearing screen after every refresh
  int i,j;
  for( i=0;i<height;i++){
    for( j=0;j<width;j++){
      if(Grid[i][j])
      printf("#");// for saving permament blocks
      else{
        int space=0;
        int x,y;
        for( x=0;x<4;x++){
          for( y=0;y<4;y++){
            if(present.shape[x][y] && i==present.y+x && j==present.x+y){
              space=1;
            }
          }
        }
        printf(space ? "*" : ".");//for present blocks and free space
      }
    }
    printf("\n");
  }
  printf("Points: %d\n", points);
}

int checkbump(int cx,int cy){     //cx-change in x ; cy-change in y
```

project.c

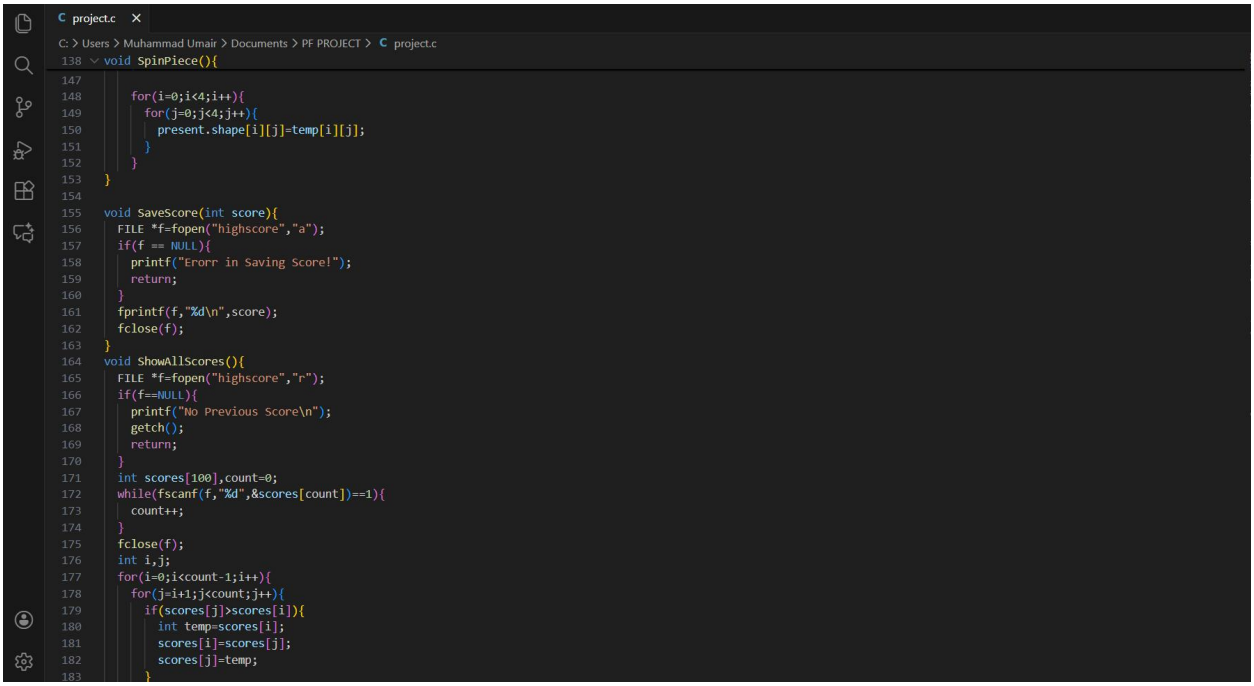
C: > Users > Muhammad Umair > Documents > PF PROJECT > project.c

```c
int i,j;
    for(i=0;i<4;i++){
      for(j=0;j<4;j++){
        if(present.shape[i][j]){
          int nx=present.x+j+cx; // nx and ny for new x and y
          int ny=present.y+i+cy;
        if(nx<0 || nx>=width || ny>=height || (ny>0 && Grid[ny][nx])){
          return 1;
        }
        }
      }
    }
    return 0;
}
void PieceFixed(){
  int i,j;
    for(i=0;i<4;i++){
      for(j=0;j<4;j++){
        if(present.shape[i][j]){
            int nx=present.x+j;
            int ny=present.y+i;
          if(ny>0){
            Grid[ny][nx]=1;
          }
        }

      }
    }
  points+=1;
}

void VanishLines(){
  int vanish=0;
  int i,j;
  for(i=0;i<height;i++){
    int fill=1;
      for (j=0;j<width;j++){
        if(!Grid[i][j]){
```

project.c

C: > Users > Muhammad Umair > Documents > PF PROJECT > project.c

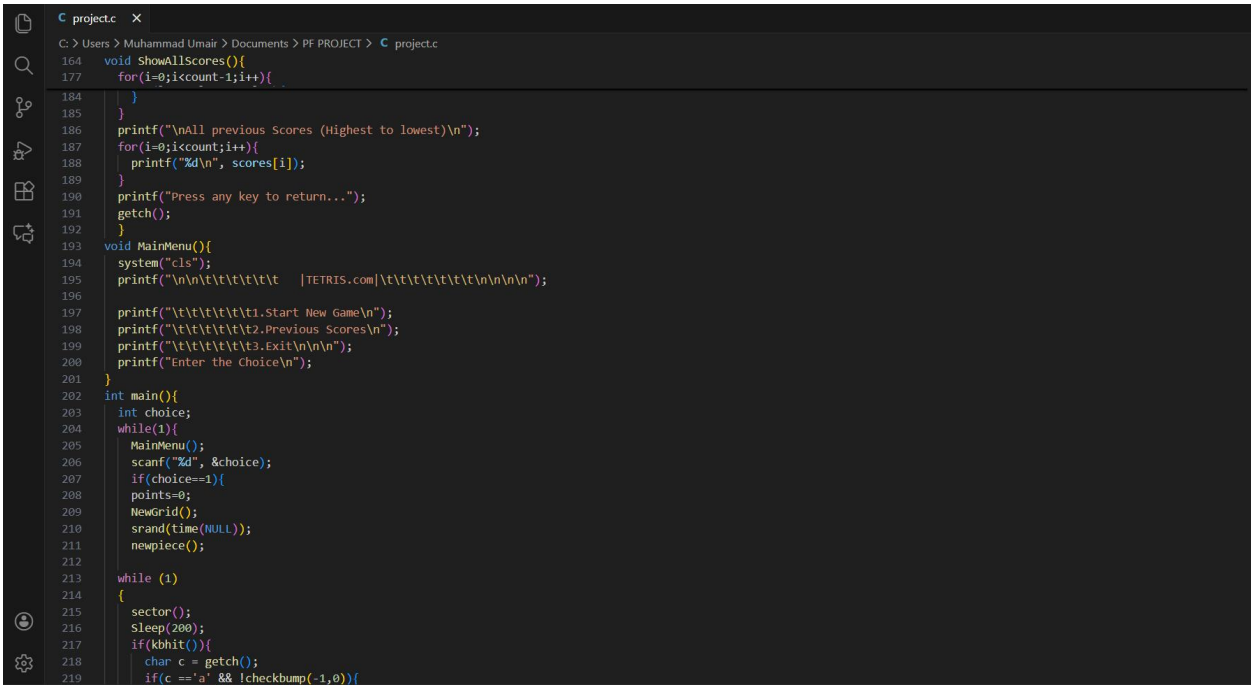
```c
void VanishLines(){
  for(i=0;i<height;i++){
    int fill=1;
      for (j=0;j<width;j++){
        if(!Grid[i][j]){
          fill=0;
          break;
        }
    }

    if(fill){
      int k,i,j;
      for(k=i;k>0;k--){
        for( j=0;j<width;j++){
          Grid[k][j]=Grid[k-1][j];
        }
      }
      for(j=0;j<width;j++){
        Grid[0][j]=0; //vanish the filled line from the grid
      }
      vanish++;
    }
  }
  if(vanish>0){
    points+=vanish*10; //adding number of vanish lines
  }
}

void SpinPiece(){
    int temp[4][4];
    int i,j;

    for( i=0;i<4;i++){
      for( j=0;j<4;j++){
        temp[j][3-i]=present.shape[i][j];
      }
    }
```



project.c

C: > Users > Muhammad Umair > Documents > PF PROJECT > project.c

```
138 void SpinPiece(){
147
148     for(i=0;i<4;i++){
149       for(j=0;j<4;j++){
150         present.shape[i][j]=temp[i][j];
151       }
152     }
153 }
154
155 void SaveScore(int score){
156   FILE *f=fopen("highscore","a");
157   if(f == NULL){
158     printf("Erorr in Saving Score!");
159     return;
160   }
161   fprintf(f,"%d\n",score);
162   fclose(f);
163 }
164 void ShowAllScores(){
165   FILE *f=fopen("highscore","r");
166   if(f==NULL){
167     printf("No Previous Score\n");
168     getch();
169     return;
170   }
171   int scores[100],count=0;
172   while(fscanf(f,"%d",&scores[count])==1){
173     count++;
174   }
175   fclose(f);
176   int i,j;
177   for(i=0;i<count-1;i++){
178     for(j=i+1;j<count;j++){
179       if(scores[j]>scores[i]){
180         int temp=scores[i];
181         scores[i]=scores[j];
182         scores[j]=temp;
183       }
```



project.c

C: > Users > Muhammad Umair > Documents > PF PROJECT > project.c

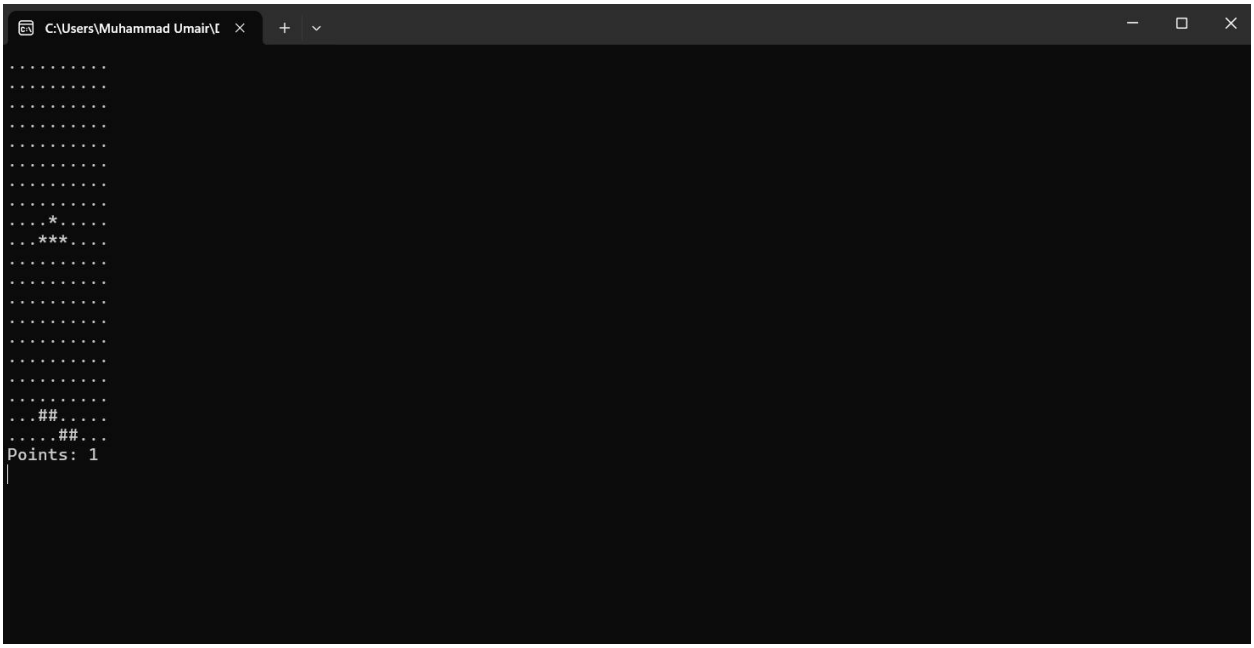
```
164 void ShowAllScores(){
177   for(i=0;i<count-1;i++){
184     }
185   }
186   printf("\nAll previous Scores (Highest to lowest)\n");
187   for(i=0;i<count;i++){
188     printf("%d\n", scores[i]);
189   }
190   printf("Press any key to return...");
191   getch();
192   }
193 void MainMenu(){
194   system("cls");
195   printf("\n\n\t\t\t\t\t\t   |TETRIS.com|\t\t\t\t\t\t\n\n\n\n");
196
197   printf("\t\t\t\t\t\t1.Start New Game\n");
198   printf("\t\t\t\t\t\t2.Previous Scores\n");
199   printf("\t\t\t\t\t\t3.Exit\n\n\n");
200   printf("Enter the Choice\n");
201 }
202 int main(){
203   int choice;
204   while(1){
205     MainMenu();
206     scanf("%d", &choice);
207     if(choice==1){
208     points=0;
209     NewGrid();
210     srand(time(NULL));
211     newpiece();
212
213   while (1)
214   {
215     sector();
216     Sleep(200);
217     if(kbhit()){
218       char c = getch();
219       if(c =='a' && !checkbump(-1,0)){
```

```c
int main(){
  {
    if(kbhit()){
      if(c =='a' && !checkbump(-1,0)){
      present.x--;}
      if(c =='d' && !checkbump(1,0)){
      present.x++;}
      if(c =='s' && !checkbump(0,1)){
      present.y++;}
      if(c =='w'){
      SpinPiece();}
    }

    if(!checkbump(0,1)){
      present.y++;
    }
    else{
      PieceFixed();
      VanishLines();
      newpiece();

      if(checkbump(0,0)){
        sector();
        printf("\n\t\t GAME OVER!  \n\t\tFINAL SCORE= %d" ,points);
        SaveScore(points);
        printf("Press any key to return\n");
        getch();
        break;
      }
    }
  }
  }
  else if(choice == 2){
    ShowAllScores();
  }
  else if(choice == 3){
  printf("Exiting..\n");
  return 0;
```

```c
int main(){
    {
        else{
        }
    }
    }
    else if(choice == 2){
        ShowAllScores();
    }
    else if(choice == 3){
    printf("Exiting..\n");
    return 0;
    }
    else{
        printf("Invalid Choice\n");
        getch();
    }
}
}
```

OUTPUT SNIPPET:

```
                                    |TETRIS.com|


                                  1.Start New Game
                                  2.Previous Scores
                                  3.Exit

Enter the Choice
```


C:\Users\Muhammad Umair\[
.....
.....
.....
.....
.....
.....
.....
.....
....*.....
...***....
.....
.....
.....
.....
.....
.....
.....
...##.....
.....##...
Points: 1


...*.....
...##*....
...###....
...###....
...##.....
...####...
...##.....
...##.....
...####...
...##.....
....#.....
...###....
...##.....
...##.....
....#.....
...###....
....#.....
...###....
...##.....
.....##...
Points: 11
GAME OVER!
FINAL SCORE= 11Press any key to return


|TETRIS.com|
1.Start New Game
2.Previous Scores
3.Exit
Enter the Choice
2
All previous Scores (Highest to lowest)
17139
158
22
19
12
11
11
11
11
11
11
11
10
1
1
1
Press any key to return...